MobileVC 后端四层架构重构方案

> 日期：2026-05-01

现有问题

当前后端按"功能模块"划分（ws/、runtime/、session/、runner/、store/），导致：

- **ws/handler.go 238KB** — WebSocket 连接、事件路由、权限决策、推送通知、session 管理全部耦合
- **跨

四层架构设计

```
┌──────────────────────────────────────────────────────────────────────────────────────────────────────────┐
│                移动端交接层                                                                                │
│            internal/gateway/                                                                              │
│  WebSocket 连接管理·  事件路由·  推送·  多连接广播                                                          │
├──────────────────────────────────────────────────────────────────────────────────────────────────────────┤
│               Session 维护层                                                                              │
│            internal/session/                                                                              │
│  会话生命周期·  状态机·  并发控制·  Resume/Recovery                                                         │
├──────────────────────────────────────────────────────────────────────────────────────────────────────────┤
│                 会话层                                                                                    │
│             internal/engine/                                                                              │
│  Claude/Codex CLI 交互·  PTY管理·  Stdio协议·  事件解析                                                     │
├──────────────────────────────────────────────────────────────────────────────────────────────────────────┤
│                数据管理层                                                                                 │
│             internal/data/                                                                                │
│  Session记录·  Skill/Memory目录·  权限规则·  推送Token                                                      │
└──────────────────────────────────────────────────────────────────────────────────────────────────────────┘
```

层级依赖规则

```
gateway ──────→ session ──────→ engine
   │            │
   └──────────────────────────┼──────→ data
                 │
  engine ──────────────┘      (engine 只输出事件，不直接访问 data)
```

- **data** 无内部依赖，纯存储抽象
- **en

各层详细设计

1. 数据管理层 `internal/data/`

职责边界： 所有持久化操作的统一入口，不包含任何业务逻辑。

接口定义：

```
// internal/data/store.go

type Store interface {
   // Session 记录
   CreateSession(ctx context.Context, title string) (SessionRecord, error)
   UpsertSession(ctx context.Context, record SessionRecord) error
   GetSession(ctx context.Context, id string) (SessionRecord, error)
   ListSessions(ctx context.Context) ([]SessionSummary, error)
   DeleteSession(ctx context.Context, id string) error
```

```
    // 投影快照（断线重连恢复用）
    SaveProjection(ctx context.Context, sessionID string, p Projection) error
    GetProjection(ctx context.Context, sessionID string) (Projection, error)

    // Skill 目录
    ListSkills(ctx context.Context) ([]SkillDef, error)
    SaveSkills(ctx context.Context, skills []SkillDef) error
    GetSkillSnapshot(ctx context.Context) (SkillCatalog, error)
    SaveSkillSnapshot(ctx context.Context, catalog SkillCatalog) error

    // Memory 目录
    ListMemories(ctx context.Context) ([]MemoryItem, error)
    SaveMemories(ctx context.Context, items []MemoryItem) error
    GetMemorySnapshot(ctx context.Context) (MemoryCatalog, error)
    SaveMemorySnapshot(ctx context.Context, catalog MemoryCatalog) error

    // 权限规则
    GetPermissionRules(ctx context.Context) (PermissionRuleSnapshot, error)
    SavePermissionRules(ctx context.Context, snapshot PermissionRuleSnapshot) error

    // 推送 Token
    SavePushToken(ctx context.Context, sessionID, token, platform string) error
    GetPushToken(ctx context.Context, sessionID string) (token, platform string, err error)
}
```

现有映射：

- inter

改动量： 主要是文件移动 + import 路径修正，接口基本不需改动。

---

2. 会话层 `internal/engine/`

职责边界： 封装与 AI CLI 的底层交互，输出标准化事件流。不关心上层状态管理，不感知 WebSocket。

接口定义：

```
// internal/engine/engine.go

type Engine interface {
    // 启动执行，通过 sink 回调输出事件
    Run(ctx context.Context, req RunRequest, sink EventSink) error

    // 向运行中的进程写入数据
    Write(ctx context.Context, data []byte) error

    // 写入权限决策（JSON via stdio）
    WritePermissionResponse(ctx context.Context, decision string) error

    // 关闭
    Close() error
}

type RunRequest struct {
    Command       string
    CWD           string
    Mode          Mode          // "pty" | "exec"
    PermissionMode string
    InitialInput   string
    SessionID      string        // 用于 —session-id 注入
    ResumeSessionID string
```

```
}

type Mode string
const (
    ModeExec Mode = "exec"
    ModePTY  Mode = "pty"
)

type EventSink func(event any)
```

扩展能力接口（类型断言获取）：

```
// 查询交互状态
type InteractiveChecker interface {
    CanAcceptInput() bool
}

// 查询当前权限请求
type PermissionState interface {
    HasPendingPermission() bool
    CurrentPermissionRequestID() string
}

// 查询 Claude Session ID
type SessionIDProvider interface {
    ClaudeSessionID() string
}

// 动态修改权限模式
type PermissionModeSetter interface {
    SetPermissionMode(mode string)
}
```

内部实现拆分：

文件	职责
`pty_engine.go`	PTY 模式引擎（当前 pty_runner.go 核心逻辑）
`exec_engine.go`	普通命令执行引擎
`codex_transport.go`	Codex App Server JSON-RPC 协议
`claude_transport.go`	Claude stdio JSON 权限协议
`parser.go`	输出流解析（ANSI、generic parser）
`shell.go`	Shell 检测与命令构建（Windows 支持）

现有映射：

- inter

改动量： 中等。pty_runner.go（86KB）需要拆分为 pty_engine + claude_transport。

---

3. Session 维护层 `internal/session/`

职责边界： 管理单个会话的完整生命周期，包括状态机、并发控制、自动恢复。这是核心调度层。

接口定义：

```
// internal/session/session.go

type Session struct {
    // 内部持有 engine.Engine + data.Store 引用
}

func New(id string, deps Dependencies) *Session
```

```
// 执行命令（非阻塞，goroutine 内运行）
func (s *Session) Execute(ctx context.Context, req ExecuteRequest, emit EventSink) error

// 发送输入
func (s *Session) SendInput(ctx context.Context, data string, meta protocol.RuntimeMeta, emit EventSink) error

// 自动 Resume + 输入（当前 SendInputOrResume 逻辑）
func (s *Session) SendInputOrResume(ctx context.Context, req ExecuteRequest, data string, meta protocol.RuntimeMeta, emit EventSink) error

// 权限决策
func (s *Session) SendPermissionDecision(ctx context.Context, decision string, meta protocol.RuntimeMeta, emit EventSink) error

// 停止当前执行
func (s *Session) Stop(emit EventSink) error

// 状态查询
func (s *Session) IsRunning() bool
func (s *Session) CanAcceptInput() bool
func (s *Session) Snapshot() Snapshot
func (s *Session) ControllerSnapshot() ControllerSnapshot

// 权限模式
func (s *Session) UpdatePermissionMode(mode string)
func (s *Session) CurrentPermissionRequestID() string

// 用户输入记录
func (s *Session) RecordUserInput(input string)

type Dependencies struct {
  Store     data.Store
  NewPTYEngine  func() engine.Engine
  NewExecEngine func() engine.Engine
}
```

内部组件：

组件	文件	职责
Controller	`controller.go`	AI 状态机（IDLE→THINKING→WAIT_INPUT→...），事件去
Manager	`manager.go`	互斥的 Engine 生命周期管理（同时只允许一个命令运行）
Resumer	`resumer.go`	自动恢复逻辑（buildDetachedResumeRequest、waitFo
PermissionRouter	`permission_router.go`	权限决策路由（deny/allow 的不同处理路径）

状态机（不变）：

```
IDLE → THINKING → RUNNING_TOOL → WAIT_INPUT
  ↑       ↑           ↑            |
  └──────────────────────────────────┘
```

现有映射：

- inter

改动量： 较大。需要将 runtime.Service 的调度逻辑与 session.Controller 正确合并，消除当前两层的冗余。

---

4. 移动端交接层 `internal/gateway/`

职责边界： 唯一对外接口，处理 WebSocket 连接、认证、事件路由、推送触发。不包含任何 AI 逻辑或持久化细节。

接口定义：

```
// internal/gateway/gateway.go

type Gateway struct {
    sessions *SessionRegistry   // sessionID → *session.Session 映射
    store    data.Store
    push     push.Service
}

// 处理 WebSocket 连接
func (g *Gateway) ServeWS(w http.ResponseWriter, r *http.Request)
```

事件路由表（客户端 action → 内部调用）：

Client Action	Gateway 处理
`exec`	→ `session.Execute()`
`input`	→ `session.SendInputOrResume()`
`ai_turn`	→ `session.Execute()` + `session.SendInput()`
`resume`	→ `session.Execute(resume=true)`
`permission`	→ `session.SendPermissionDecision()`
`review`	→ `session.ReviewDecision()`
`plan`	→ `session.PlanDecision()`
`stop`	→ `session.Stop()`
`create_session`	→ `store.CreateSession()`
`list_sessions`	→ `store.ListSessions()`
`delete_session`	→ `store.DeleteSession()`
`slash_command`	→ 解析后路由到对应 action
`adb_*`	→ ADB 专用处理

内部组件：

组件	文件	职责
Router	`router.go`	客户端事件分发（action dispatch）
SessionRegistry	`registry.go`	sessionID → Session 映射，多连接广播，延迟释放
PushHelper	`push.go`	推送通知触发（去重、连接状态检查）
SlashCommands	`slash.go`	Slash command 解析与路由
ADBHandler	`adb.go`	ADB 设备管理、WebRTC 投屏

现有映射：

- inter

改动量： 最大。需要将 238KB 的 handler.go 拆分为 6 个文件，并移除其中的业务逻辑到 session 层。

公共模块 `internal/protocol/`

保持不变，但建议拆分：

文件	内容
`event.go`	服务端→客户端事件类型定义
`request.go`	客户端→服务端请求类型定义

`meta.go`	`RuntimeMeta` + `ApplyRuntimeMeta()` + `MergeRuntimeMeta()`
`event_cursor.go`	`ApplyEventCursor()` + cursor 提取

当前 event.go 55KB，包含所有事件 + 客户端请求 + RuntimeMeta + 辅助函数，按上述拆分后可维护性大幅提升。

目录结构对比

当前

```
internal/
├── ws/        (238KB handler.go + 多文件，耦合严重)
├── runtime/    (35KB manager.go + service + info)
├── session/    (20KB controller)
├── runner/     (86KB pty_runner + exec_runner + codex)
├── store/      (33KB file_store + interface)
├── protocol/   (55KB event.go)
├── adapter/    (ANSI parser)
├── claudesync/  (JSONL sync)
├── codexsync/   (Codex sync)
├── skills/     (skill launcher)
├── push/       (APNs)
├── config/
├── tts/
├── adb/
└── logx/
```

重构后

```
internal/
├── gateway/        # 移动端交接层
│   ├── gateway.go   (ServeHTTP 入口)
│   ├── router.go    (action dispatch)
│   ├── registry.go  (session 注册表)
│   ├── push.go      (推送触发)
│   ├── slash.go     (slash command)
│   └── adb.go       (ADB 投屏)
│
├── session/        # Session 维护层
│   ├── session.go   (对外接口)
│   ├── controller.go (AI 状态机)
│   ├── manager.go   (Engine 生命周期)
│   ├── resumer.go   (自动恢复)
│   ├── permission_router.go
│   ├── permission_rules.go
│   └── projector.go (投影快照管理)
│
├── engine/         # 会话层 (AI CLI 交互)
│   ├── engine.go    (接口定义)
│   ├── pty_engine.go (PTY 模式)
│   ├── exec_engine.go(Exec 模式)
│   ├── claude_transport.go  (Claude stdio JSON)
│   ├── codex_transport.go   (Codex JSON-RPC)
│   ├── parser.go    (ANSI + generic parser)
│   └── shell.go     (Shell 检测/命令构建)
│
├── data/           # 数据管理层
│   ├── store.go     (Store 接口)
│   ├── file_store.go (JSON 文件实现)
│   ├── claudesync/  (JSONL 同步)
│   ├── codexsync/   (Codex 同步)
│   └── skills/      (Skill 管理)
│
├── protocol/       # 公共协议
```

```
│   ├── event.go    (服务端事件)
│   ├── request.go  (客户端请求)
│   ├── meta.go     (RuntimeMeta)
│   └── cursor.go   (事件游标)
│
├── push/        # APNs (不变)
├── config/      # 配置 (不变)
├── tts/         # TTS (不变)
├── adb/         # ADB (不变，部分逻辑上提到 gateway)
└── logx/        # 日志 (不变)
```

分步实施计划

阶段 1：数据层独立（改动最小，风险最低）

1. internal/store/ → internal/data/ 移动
2. inte

阶段 2：引擎层独立

1. 创建 internal/engine/engine.go 定义接口
2. inte

阶段 3：Session 层重构

1. 合并 session.Controller + runtime.Service + runtime.manager 为统一的 session.Session
2. 提取

阶段 4：Gateway 层拆分

1. 拆分 ws/handler.go 为 router.go + registry.go + push.go + slash.go + adb.go
2. 移除

阶段 5：清理

1. 删除旧包（internal/ws/、internal/runtime/、internal/runner/、internal/store/、internal/adapter/）
2. 更新

关键设计决策

1. Engine 不感知上层

Engine 只通过 EventSink 回调输出事件，不知道事件最终会被状态机处理还是直接发给客户端。这让 Engine 可以独立测试。

2. Session 持有 Engine 和 Store

Session 是"粘合层"——它从 Engine 接收事件，经过 Controller 状态机处理后输出，同时通过 Store 持久化投影快照。

3. Gateway 不包含业务逻辑

Gateway 只做：认证 → 反序列化 → 路由到 Session 方法 → 序列化返回。权限决策、Resume 判断等全部在 Session 层处理。

4. 多连接广播在 Gateway 的 Registry 中

SessionRegistry 维护 sessionID → []listener 映射，同一个 Session 可以被多个 WebSocket 连接监听，实现手机+手表同时接收事件。

风险与缓解

风险	缓解
大量文件移动导致合并冲突	阶段式进行，每阶段独立 commit
接口设计不合理	先用 adapter 模式保留旧接口，新接口稳定后再删除
测试大量失败	每阶段完成后跑全量测试，不累积
handler.go 拆分引入回归	保持每个新文件不超过 1000 行，逐文件迁移